

Rapport final RLMinimax

Projet Pac-Man - Algorithmique 4 - 2025/2026

Poids appris, resultats finaux, justification algorithmique et controle de conformite au sujet.

Statut global	Conclusion
Conforme au sujet technique	Le projet implemente un agent RLMinimax : Alpha-Beta + fonction affine + poids appris par renforcement + comparaison statistique.
Verification du 14/05/2026	Syntaxe Python OK, formule affine verifiee, poids coherents avec les 7 features, comparaisons finales relancees.

Point administratif a verifier separement : le dossier local ne contient pas de depot Git visible ici. Les commits/pushs sur Gitesi doivent etre confirmes dans le vrai depot de remise.

1. Conformite au sujet

Le PDF du projet demande explicitement : des facteurs lies a Pac-Man, une fonction affine, un agent Minimax avec elagage Alpha-Beta, des poids appris par renforcement et une comparaison statistique avec Minimax.

Exigence du sujet	Element dans le projet	Statut
Facteurs $x_i(s)$ lies a Pac-Man	7 features dans features.py : nourriture, fantomes, capsules, score, nourriture restante.	OK
Recherche dans certains facteurs	BFS utilise pour distances reelles vers nourriture, capsules et fantomes.	OK
Fonction affine $f(s)$	rl_evaluation_function calcule $\sum(a_i * x_i(s)) + C$, sans termes croises.	OK
Poids independants de l etat	weights.json contient les poids fixes utilises par RLMinimax.	OK
Minimax avec Alpha-Beta	RLMinimaxAgent explore les successeurs avec alternance Pac-Man MAX / fantomes MIN et coupure $\alpha \geq \beta$.	OK
Poids appris par renforcement	train.py simule des parties et ajuste les poids par mise a jour TD.	OK
Pas une politique directe apprise	Le RL apprend les parametres de l evaluation ; l action finale vient d Alpha-Beta.	OK
Comparaison statistique	compare.py mesure score moyen, winrate, temps, coups et Stop moyen.	OK

Choix de conception : l action Stop est ignoree quand Pac-Man dispose d un mouvement reel. Cela evite un comportement immobile et garde la recherche centree sur les actions strategiques utiles.

2. Fonction affine et poids actuels

La fonction d'évaluation utilisée par RLMinimax respecte la forme imposée par le sujet :

$$f(s) = a_1 \cdot x_1(s) + a_2 \cdot x_2(s) + \dots + a_7 \cdot x_7(s) + C$$

Facteur	Fonction	Poids	Information	Interpretation
x1	nearest_food_bfs	0.6891	Nourriture proche	Positif : Pac-Man cherche la nourriture.
x2	ghosts_within_3	-1.4093	Fantômes dangereux proches	Négatif : Pac-Man évite le danger.
x3	scared_ghosts_nearby	1.4061	Fantômes effrayés proches	Positif : Pac-Man profite des opportunités.
x4	remaining_food	0.4036	Nourriture restante	Positif sur une valeur négative : finir la carte est encouragé.
x5	nearest_capsule_bfs	0.0252	Capsule proche	Positif faible : utile, mais moins dominant.
x6	current_score	1.8572	Score courant normalisé	Positif fort : respecter le score officiel.
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal proche	Négatif : rester loin des fantômes dangereux.
C	bias	-0.0834	Correction globale	Constante indépendante de l'état.

Vérification automatique : 7 poids pour 7 features, et la valeur calculée par le code est identique à la somme manuelle $\sum(a_i \cdot x_i) + C$.

3. Resultats finaux relances

Les resultats ci-dessous ont ete relances le 14/05/2026 avec les poids actuels, sans reentrainement. compare.py utilise seed=0 par default, donc les agents affrontent les memes fantomes dans le meme ordre.

Layout	Depth	N	Minimax	AlphaBeta	RLMinimax	RL-Minimax	RL-AlphaBeta	Win RL	Temps RL	Stop RL
testClassic	1	200	548.5	548.5	552.5	+4.0	+4.0	100%	0.016s	0.0
smallClassic	2	100	-149.8	-149.8	196.6	+346.4	+346.4	34%	0.284s	0.0
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7	+93.7	12%	0.304s	0.0
mediumClassic	1	50	14.5	14.5	515.5	+501.0	+501.0	28%	0.219s	0.0
originalClassic	1	50	348.6	348.6	664.4	+315.7	+315.7	2%	1.430s	0.0
trickyClassic	1	50	101.3	101.3	293.0	+191.7	+191.7	2%	0.643s	0.0
contestClassic	1	50	11.3	11.3	244.4	+233.1	+233.1	28%	0.214s	0.0
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4	+90.4	34%	0.036s	0.0
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9	+0.9	11%	0.001s	0.0
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5	+1802.4	90%	0.521s	0.0

Lecture : RLMinimax bat Minimax et AlphaBeta en score moyen sur tous les layouts testes. Le winrate n est pas toujours parfait, mais le score moyen montre que le comportement global est meilleur dans ces conditions.

4. Justification algorithmique

Point	Justification
Pourquoi BFS ?	Le sujet attend des facteurs pouvant faire de la recherche. BFS donne une distance réelle dans le labyrinthe, plus pertinente qu'une simple distance Manhattan quand il y a des murs.
Pourquoi Alpha-Beta ?	Alpha-Beta donne le même choix théorique que Minimax à profondeur équivalente, mais évite d'explorer des branches inutiles grâce aux bornes alpha et beta.
Pourquoi une fonction affine ?	Le sujet impose une évaluation de type somme pondérée des facteurs. Le modèle reste simple, explicable et compatible avec l'apprentissage des poids.
Pourquoi apprendre les poids ?	Les poids ne sont pas choisis manuellement : train.py simule des parties et ajuste les coefficients selon les récompenses observées.
Pourquoi comparer ?	La comparaison avec Minimax et AlphaBeta montre si les poids appris apportent une amélioration mesurable.

Limites honnêtes : les poids ne garantissent pas une victoire sur toutes les cartes possibles, et openClassic est un cas particulier car les baselines dépassent le timeout choisi. Ces limites sont normales et peuvent être expliquées à l'oral.

5. Commandes reproductibles

But	Commande
Verifier la syntaxe	<code>python -m py_compile compare.py game.py pacman.py layout.py train.py rlMinimaxAgents.py features.py multiAgents.py ghostAgents.py textDisplay.py util.py</code>
Comparer sur smallClassic	<code>python compare.py --num-games 100 --layout smallClassic --depth 2 --baseline both</code>
Relancer RLMinimax	<code>python pacman.py -p RLMinimaxAgent -l testClassic -a depth=1 -q -n 1 -f</code>
Voir les poids	<code>weights.json</code>

Conclusion finale

Le correctif est termine pour la partie technique du projet : l'agent RLMinimax respecte le sujet, les poids actuels sont coherents, les resultats finaux sont documentes et les tests ont ete relances. Le seul point a confirmer hors code est le depot Git/Gitesi, car il depend du depot de remise.